Reporte

Nombre:	**Matrícula**:
Eduardo Alejandro Acevedo Liuskos	2142139
Emma Patricia Brown Sánchez	2135479
Diego Monroy García	2135198

Nombre del curso: Componentes de lógica de negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 3	**Actividad**: Diseño arquitectural del sistema con IA — TecnoStock (Caso 05)
Fecha: 12/10/2026	

Bibliografia (Formato APA)

- Documento de caso asignado enviado por el docente. (2026). Caso 05: TecnoStock, retail de electrónica.
- Acevedo Liuskos, E. A., Brown Sánchez, E. P., & Monroy García, D. (2026). BRD de TecnoStock [Documento del curso Componentes de lógica de negocio].
- Acevedo Liuskos, E. A., Brown Sánchez, E. P., & Monroy García, D. (2026). User Stories de TecnoStock [Documento del curso Componentes de lógica de negocio].
- Evans, E. (2003). Domain-Driven Design: Tackling complexity in the heart of software. Addison-Wesley.
- Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.
- Martin, R. C. (2017). Clean architecture: A craftsman's guide to software structure and design. Prentice Hall.
- Percival, H., & Gregory, B. (2020). Architecture patterns with Python. O'Reilly Media.

Objetivo

Proponer una arquitectura de software coherente para TecnoStock a partir del BRD y de las User Stories de la Unidad 2: seleccionar cinco casos de uso, justificar el patrón de lógica de negocio con evidencia, modelar el dominio, diseñar la Service Layer con Repository, Unit of Work y CQRS básico, revisar el diseño con SOLID y representarlo en UML y ADR.

Procedimiento

Se revisaron las 20 User Stories y el catálogo de reglas del BRD; se eligieron cinco casos de uso con flujos normales, rechazos y casos distintos; se compararon Transaction Script, Table Module y Domain Model; se construyó el mapa de dominio y se diseñaron servicios, repositorios y consultas; se corrigió el diseño con SOLID y se documentaron las decisiones. No se desarrolló base de datos.

1. Alcance y continuidad con la Unidad 2

Este informe convierte el análisis de la Unidad 2 (BRD y 20 User Stories de TecnoStock) en una propuesta de arquitectura que separa dominio, aplicación e infraestructura. No se abre un caso nuevo: todas las decisiones se apoyan en los identificadores del BRD (BR-###, RN-XXX-###) y de las historias (US##). Los valores marcados como supuesto en el BRD (anticipo de 20%–30%, vigencia de 7 días, devolución de 15–30 días, conciliación mensual) se tratan como parámetros configurables y no como constantes del diseño.

1.1 Proceso principal elegido

Apartado, venta de unidades serializadas y devolución con autorización del Supervisor. Es el proceso donde el BRD concentra los problemas observados: venta accidental de productos apartados, trazabilidad de números de serie y criterios de devolución inconsistentes. Quedan fuera del diseño detallado, por ser de menor prioridad o independientes, las transferencias entre sucursales (BR-007, US18–US19), la conciliación de inventario (BR-008, US15) y el alta de unidades (US17); el modelo de dominio los deja previstos sin desarrollarlos.

1.2 Hallazgos sobre la Unidad 2

Según la regla de continuidad, las contradicciones o datos faltantes se registran como hallazgos y no se resuelven inventando reglas. Cada hallazgo indica cómo lo trata el diseño y la pregunta que debe validarse con el stakeholder.

ID	Hallazgo	Evidencia	Tratamiento en el diseño / pregunta pendiente
H-01	El documento de historias contiene 20 US (el mínimo exigido era 15). El actor «Sistema» de US20 no aparece entre los cinco stakeholders del BRD.	US01–US20; BRD, Stakeholders	Se trabaja con las 20 US. «Sistema» se trata como actor técnico (temporizador), no como rol de negocio. Sin cambio de alcance.
H-02	BR-005 pide validar «estado del producto y causal», pero el catálogo solo tiene RN-DEV-001 (comprobante) y RN-DEV-002 (plazo).	BR-005; RN-DEV-001/002	No se inventa regla. El criterio de devolución es una lista de ReglaDeDevolucion extensible. Pregunta: ¿qué condiciones de estado y causal existen?
H-03	No se define qué pasa con el anticipo cuando el apartado vence o se cancela.	US14, US20; RN-APT-002	El diseño solo libera la unidad y registra el hecho. Pregunta: ¿el anticipo se reembolsa, se retiene o se abona?
H-04	US01 habla de «reunir el pago	US01, US07	Solo se modela la liquidación del saldo

ID	Hallazgo	Evidencia	Tratamiento en el diseño / pregunta pendiente
	completo» y US07 de «pagar el saldo restante», pero el BRD no define abonos parciales.		completo en una operación. Pregunta: ¿se permiten abonos intermedios?
H-05	No se indica si el precio se congela al apartar.	US07 («saldo pendiente»)	Para que el saldo tenga significado, Apartado guarda el precio acordado. Supuesto pendiente de validar.
H-06	No se define el destino físico de una unidad con devolución aceptada.	BR-003, BR-005	Se usa el estado neutro «Devuelta» (pendiente de revisión por Almacén). Pregunta: ¿cuándo vuelve a ser Disponible?
H-07	No se aclara si una devolución sin comprobante puede autorizarla el Supervisor.	RN-DEV-001 vs. RN-SUP-002	Sin comprobante se rechaza de forma definitiva; solo el incumplimiento de criterio se deriva al Supervisor. Pregunta pendiente.
H-08	No se definen los datos del Cliente ni cómo se controla el stock de productos no serializables (accesorios).	BRD, Actores; RN-VTA-001	Cliente se maneja solo como ClienteId. Las piezas no serializables siguen siendo UnidadInventario sin número de serie; el almacenamiento agregado queda en infraestructura. Pendiente de validar.

2. Preparación del caso: selección de casos de uso (Etapa 1)

2.1 Criterio de selección

De las 20 historias se eligieron cinco casos de uso que cumplen simultáneamente tres condiciones: (a) forman una cadena de negocio continua, de la reserva del producto a su devolución; (b) incluyen operaciones normales y operaciones que el sistema puede **rechazar**; y (c) después del rechazo el negocio **toma un camino distinto** y no simplemente termina. Además tocan los seis aggregates principales, por lo que ponen a prueba la arquitectura completa.

ID	Caso de uso	Actor	Reglas relacionadas	Resultado esperado
CU-01	Apartar producto con anticipo	Vendedor / Cajero	RN-APT-001*, RN-APT-002*, RN-APT-003, RN-VTA-002	Apartado vigente, unidad bloqueada y confirmación con fecha límite
CU-02	Vender producto serializado	Cajero (con Vendedor)	RN-VTA-001, RN-VTA-002, RN-APT-003, RN-INV-002	Venta cerrada con la serie exacta y

ID	Caso de uso	Actor	Reglas relacionadas	Resultado esperado
				comprobante con folio único
CU-03	Convertir apartado vigente en venta	Vendedor / Cajero	RN-APT-002*, RN-APT-003, RN-VTA-001, RN-VTA-002	Venta cerrada aplicando el anticipo, o apartado vencido y unidad liberada
CU-04	Solicitar devolución	Cajero (con Vendedor)	RN-DEV-001, RN-DEV-002*	Devolución aceptada, o en revisión si incumple el criterio estándar
CU-05	Autorizar devolución excepcional	Supervisor	RN-SUP-002, RN-SUP-001	Devolución aceptada o rechazada con autorización documentada

* Regla marcada como supuesto en el BRD; su valor es configurable.

Información de entrada de cada caso de uso (la guía pide actor, objetivo, entrada, reglas y resultado):

ID	Objetivo del actor	Información de entrada
CU-01	Reservar una unidad pagando un anticipo.	Producto, sucursal, cliente, anticipo (método y monto).
CU-02	Entregar y cobrar una unidad identificada.	Sucursal, líneas (producto y serie capturada), pagos.
CU-03	Completar la compra de un producto apartado.	Folio del apartado, pago del saldo, serie capturada.
CU-04	Devolver un producto comprado.	Folio de venta (comprobante), producto, serie, motivo.
CU-05	Resolver una devolución fuera de criterio.	Folio de la devolución, decisión (autoriza o deniega), motivo.

Comprobación contra las historias de usuario

La guía exige que cada caso de uso aparezca en las historias de usuario o en el BPMN. Los cinco aparecen en las US; ninguno requiere corrección de alcance.

ID	US de origen	Necesidades del BRD	¿Aparece en las US?
CU-01	US01, US03, US11, US12	BR-004, BR-002	Sí (criterios Gherkin US01, US03, US11)
CU-02	US04, US06, US09, US12	BR-001, BR-002, BR-003	Sí (US06, US09)
CU-03	US07, US12; vencimiento de	BR-004, BR-003	Sí (US07, US20)

ID	US de origen	Necesidades del BRD	¿Aparece en las US?
	US20		
CU-04	US10, US08, US12 (consulta US02)	BR-005, BR-003	Sí (US10, US08)
CU-05	US13 (consulta US16)	BR-006, BR-005	Sí (US13)

2.2 Operaciones normales, rechazos y casos distintos

Caso de uso	Flujo normal	Rechazo posible	Caso distinto que toma el negocio
CU-01	Hay unidad disponible y el anticipo cumple el mínimo: se crea el apartado.	Sin existencia local; anticipo menor al mínimo; método de pago no autorizado.	Sin existencia: consultar otras sucursales (US05). Anticipo bajo: reintentar con el mínimo.
CU-02	Se escanea la serie de una unidad disponible y se cobra con método autorizado.	Unidad apartada y vigente (US06); serie no capturada (RN-VTA-001); pago inválido.	Elegir otra unidad disponible, capturar la serie, o consultar otras sucursales.
CU-03	Apartado vigente y saldo liquidado: se crea la venta con el anticipo como parte del pago.	Apartado vencido (RN-APT-002); saldo insuficiente; método no autorizado.	Vencido: la unidad se libera y se inicia una venta nueva (CU-02) si hay existencia.
CU-04	Comprobante válido y dentro de criterio: devolución aceptada.	Comprobante inexistente (definitivo); fuera de plazo (derivable).	Fuera de plazo: la devolución queda «en revisión» y pasa a CU-05.
CU-05	El Supervisor autoriza: devolución aceptada con su autorización registrada.	El operador no es Supervisor; la devolución no está en revisión.	El Supervisor puede denegar: devolución rechazada con motivo documentado.

2.3 Tres tipos de rechazo

El análisis de los flujos muestra que «rechazar» no significa lo mismo en todos los casos, y esa diferencia condiciona la arquitectura (ADR-006):

Tipo	Ejemplo	Qué ocurre con el estado	Cómo se implementa
Rechazo definitivo	Unidad apartada (CU-02); comprobante inexistente (CU-04)	No se guarda nada; se hace rollback.	Excepción ReglaDeNegocioViolada traducida a Rechazo.
Rechazo con derivación	Devolución fuera de plazo (CU-04)	Se guarda la devolución en estado EnRevision.	Resultado válido del dominio, no es un error.
Rechazo con efecto	Apartado vencido al convertirlo (CU-03)	El vencimiento y la liberación de la unidad sí se	ResultadoDeConversion rechazada y

Tipo	Ejemplo	Qué ocurre con el estado	Cómo se implementa
		guardan.	Confirmar() del UoW.

2.4 Historias no incluidas en el diseño detallado

US	Motivo
US04, US05, US02, US16	Son consultas: se diseñan como Queries (sección 8), no como casos de uso de escritura.
US14 (cancelar apartado)	Comparte el comportamiento de Apartado (método Cancelar) y de UnidadInventario (LiberarApartado); no añade decisiones nuevas.
US15, US17, US18, US19	Conciliación, alta y transferencias pertenecen a la gestión de inventario; el aggregate UnidadInventario ya prevé sus estados.
US20 (liberación por vencimiento)	El comportamiento se diseña dentro de CU-03 (Apartado.Vencer). Su disparo programado reutiliza el mismo método.
US08, US12	Se integran en CU-04 (motivo) y como efecto de eventos (comprobante), no como casos de uso propios.

3. Comparación de patrones de lógica de negocio (Etapa 2)

3.1 Tabla comparativa

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
Complejidad de reglas	Un procedimiento por operación. Con 15 reglas RN cruzadas se duplican validaciones entre scripts (p. ej. disponibilidad en venta, apartado y transferencia).	Reglas agrupadas por tabla; las reglas que cruzan Apartado, Unidad y Venta quedan sin dueño.	Cada regla vive en el objeto que protege: anticipo en Apartado, serie y estado en UnidadInventario, criterio en CriterioDeDevolucion.	RN-VTA-001/002, RN-APT-001/002/003, RN-INV-002/003, RN-DEV-001/002, RN-SUP-001/002
Cambios esperados	Cada cambio de valor o política obliga a editar scripts completos.	Cambios acotados a una clase por tabla, pero no a una política.	Los valores supuestos son parámetros de PoliticaComercial y las reglas nuevas son nuevas clases ReglaDeDevolucion.	Supuestos: 20%–30%, 7 días, 15–30 días por categoría; US16 (criterio vigente definido por el Supervisor)
Interacción	Se coordina a	Un módulo por	Aggregates colaboran por	US07 (apartado +

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
entre conceptos	mano en el script; el riesgo es olvidar un paso.	tabla; la coordinación queda fuera.	identidad; el servicio de dominio ConversionDeApartado coordina los tres que intervienen.	unidad + venta + anticipo), US06, US13
Estados e invariantes	Los estados son columnas; cualquier script puede dejar un estado inválido.	Ídem: nada impide la transición incorrecta.	Los estados tienen transiciones controladas por el aggregate (Vigente→Convertido, Vencido o Cancelado; Disponible→Apartada→Vendida).	RN-APT-002/003, RN-INV-003; problema: venta accidental de apartados
Criterio adicional: trazabilidad por unidad y autorización por rol	La serie es un dato más y la unicidad depende de cada script; el rol se verifica en cada script.	Tabla de series con consultas; el rol se verifica en el módulo.	UnidadInventario tiene identidad y ciclo de vida (venta, tránsito, devolución); DecisionDeSupervisor solo se crea con rol Supervisor.	BR-003, RN-INV-002, US09, US17; RN-SUP-001/002

Ajuste global al caso: Transaction Script, bajo (sirve para lecturas simples); Table Module, bajo a medio (sirve para catálogos); Domain Model, **alto** para el núcleo de apartado, venta y devolución.

3.2 Ventajas, riesgos y consecuencias en TecnoStock

Alternativa	Ventajas	Riesgos	Consecuencia para TecnoStock
Transaction Script	Rápido de construir; fácil de leer con pocas reglas.	Reglas duplicadas; las transiciones de estado se vuelven condicionales dispersos; difícil probar el bloqueo de apartados.	Reproduciría el problema observado: la venta accidental de apartados aparece cuando la verificación no está en un solo lugar.
Table Module	Buena correspondencia con tablas; simple para catálogos.	Las reglas que cruzan tablas no tienen dueño; la identidad de la unidad se diluye.	Adecuado solo para consultas y para el catálogo, no para el núcleo.
Domain Model	Invariantes protegidas por el modelo; lenguaje ubicuo; pruebas unitarias sin infraestructura; reglas parametrizables.	Mayor esfuerzo inicial; requiere disciplina para no mover reglas a los servicios; exige mapeo de persistencia.	El costo se justifica por el volumen de reglas cruzadas y por los estados del apartado y de la unidad.

3.3 Decisión

Se selecciona **Domain Model como patrón principal** para las operaciones que cambian el estado (CU-01 a CU-05), combinado con **Transaction Script en el lado de lectura** (consultas directas que devuelven DTO, sección 7). La combinación queda delimitada: el Domain Model decide y protege; los lectores solo proyectan datos. Table Module se descarta.

Evidencias de la Unidad 2 que respaldan la decisión:

- **Estados con ciclo de vida (RN-APT-002, RN-APT-003, RN-INV-003).** El apartado cambia de vigente a vencido por el paso del tiempo y bloquea la unidad; la unidad pasa por disponible, apartada, en tránsito, vendida. Esas transiciones necesitan dueño.
- **Interacción entre conceptos (US07, US06, US13).** Convertir un apartado involucra el apartado, la unidad reservada, el anticipo y la venta; rechazar una venta depende del estado de otro aggregate.
- **Parámetros en rangos y por categoría (supuestos del BRD, US16, RN-VTA-002 «política vigente»).** Los valores aún no están cerrados y el Supervisor define el criterio; el diseño debe absorber el cambio sin reescribir lógica.
- **Problemas observados (BRD, Problema y Oportunidad).** Venta accidental de productos apartados y falta de trazabilidad de series exigen invariantes protegidas en un solo lugar («bloqueo atómico de stock apartado»).

Costo aceptado. El Domain Model introduce más clases, requiere traducir persistencia a agregados y obliga al equipo a mantener las reglas en el dominio. Se acepta porque el costo de una venta indebida o de una serie mal rastreada es mayor que el costo de construcción; para el catálogo y las consultas se evita el sobrediseño usando lecturas directas.

4. Mapa de dominio (Etapa 3)

El mapa identifica lo que el sistema debe reconocer (entidades), proteger (invariantes) y comunicar (eventos). No se convirtió cada tabla en entidad: Cliente, Sucursal y Comprobante, por ejemplo, no son aggregates (ver 4.7). Convención: los identificadores en código van sin acentos (Almacen, Devolucion).

4.1 Lenguaje ubicuo

Término	Significado en TecnoStock	Fuente	Estado
Apartado	Reserva de una unidad mediante anticipo, con	BR-004, RN-APT	Existente

Término	Significado en TecnoStock	Fuente	Estado
	vigencia y bloqueo de la unidad.		
Anticipo	Pago inicial que formaliza el apartado; no puede ser menor al mínimo de la política.	US11, RN-APT-001*	Existente
Saldo pendiente	Precio acordado menos el anticipo; se liquida para convertir el apartado en venta.	US07	Ajustado (H-05)
Vigencia / vencido	Plazo máximo del apartado; al cumplirse sin liquidar pasa a «vencido».	RN-APT-002*	Existente
Unidad de inventario	Pieza física con identidad propia y estado; su número de serie es obligatorio si el producto es serializable.	BR-003, RN-INV-002	Nuevo en el diseño
Número de serie	Identificador único de una unidad activa; no se duplica.	RN-INV-002, US17	Existente
Producto serializable	Producto cuya unidad exige número de serie al vender.	RN-VTA-001	Existente
Disponible / Apartada / Vendida / En tránsito / Devuelta	Estados de la unidad. «Devuelta» es provisional hasta definir H-06.	RN-APT-003, RN-INV-003	Nuevo (Devuelta)
Venta	Operación de cobro en sucursal que entrega unidades identificadas.	BR-002, US09	Existente
Método de pago autorizado	Forma de cobro permitida por la política vigente.	RN-VTA-002	Existente
Comprobante / Folio	Documento de respaldo con folio único; el folio también identifica la operación.	US12, RN-DEV-001	Existente
Devolución	Solicitud de regreso de una unidad vendida, sujeta al criterio de devolución.	BR-005	Existente
Criterio de devolución	Conjunto de condiciones estándar (comprobante, plazo) definido por el Supervisor.	US16, RN-DEV	Existente
Dictamen	Resultado de evaluar una devolución contra el criterio: cumple o lista de incumplimientos.	US10	Nuevo
En revisión	Estado de una devolución que incumple el criterio y espera al Supervisor.	US02, US10	Existente
Autorización del Supervisor	Decisión documentada (usuario, motivo, fecha) sobre una excepción.	RN-SUP-001/002	Existente
Operador / Rol	Empleado que ejecuta la operación: Vendedor, Cajero, Supervisor o Almacén.	BRD, Stakeholders	Existente

Término	Significado en TecnoStock	Fuente	Estado
Política comercial	Valores vigentes de anticipo, plazo, métodos de pago y criterio de devolución.	Supuestos BRD, US16	Nuevo
Sucursal	Una de las cinco tiendas físicas.	BRD, Restricciones	Existente
Transferencia / Conciliación	Traslado entre sucursales y comparación físico vs. sistema (fuera del diseño detallado).	BR-007, BR-008	Existente

* Supuesto del BRD.

4.1.1 Elementos que se identificaron

Elemento	Pregunta que debe responder el equipo	Respuesta en TecnoStock
Lenguaje ubicuo	Términos usados por usuarios, analistas y desarrolladores con el mismo significado.	20 términos del glosario (4.1); identificadores de código sin acentos.
Entidades	Conceptos con identidad y ciclo de vida que deben distinguirse aun cuando cambien sus datos.	Apartado, UnidadInventario, Venta, Devolucion (raíces) y LineaDeVenta.
Value Objects	Conceptos definidos por sus valores y reglas, sin identidad propia relevante.	Dinero, NumeroDeSerie, Folio, PagoRecibido, VigenciaDeApartado, MotivoDeDevolucion, DictamenDeDevolucion, DecisionDeSupervisor, Operador.
Aggregates	Grupos que requieren un límite de consistencia y una raíz para controlar cambios.	Apartado, UnidadInventario, Venta, Devolucion, PoliticaComercial y Producto (referencia).
Domain Services	Comportamientos del dominio que no pertenecen naturalmente a una sola entidad.	ConversionDeApartado (coordina Apartado, UnidadInventario y Venta).
Domain Events	Hechos del negocio que ya ocurrieron y pueden interesar a otros componentes.	ApartadoCreado, ApartadoVencido, VentaCerrada, DevolucionDerivadaASupervisor, DevolucionAceptada, DevolucionRechazada.

4.2 Aggregates: fichas

Aggregate / raíz	Contiene	Responsabilidades	Invariantes	Eventos
Apartado	VigenciaDeApartado, anticipo	Crear con anticipo confirmado;	Anticipo ≥ mínimo (RN-	ApartadoCreado, ApartadoVencido

Aggregate / raíz	Contiene	Responsabilidades	Invariantes	Eventos
	(PagoRecibido), precio acordado, referencia a la unidad	calcular saldo; liquidar; vencer; cancelar	APT-001*); método de pago autorizado; vigencia ≤ máximo (RN-APT-002*); transiciones solo desde Vigente; cancelar exige Supervisor (RN-SUP-001)	
UnidadInventario	NumeroDeSerie, EstadoDeUnidad, sucursal actual, folio de apartado vigente, versión	Apartar, liberar, vender, registrar devolución	Solo una unidad Disponible se aparta o se vende (RN-APT-003, US06); venta solo en la sucursal donde está; serie obligatoria si es serializable (RN-VTA-001)	Ninguno propio
Venta	LineaDeVenta, pagos (PagoRecibido), folio, cajero, folio de apartado origen	Cerrar la venta; calcular total; ubicar la línea a devolver	Serie capturada en productos serializables (RN-VTA-001); método autorizado (RN-VTA-002); pagos cubren el total	VentaCerrada
Devolucion	MotivoDeDevolucion, DictamenDeDevolucion, DecisionDeSupervisor, estado	Solicitar según dictamen; resolver por decisión del Supervisor	Sin comprobante válido no existe (RN-DEV-001); incumplimiento no puede quedar Aceptada sin autorización (RN-SUP-002); solo se resuelve desde EnRevision	DevolucionAceptada, DevolucionDerivadaASupervisor, DevolucionRechazada
PoliticaComercial	Porcentaje mínimo de anticipo, plazo de	Entregar los valores vigentes y	Una sola política vigente;	Ninguno

Aggregate / raíz	Contiene	Responsabilidades	Invariantes	Eventos
	apartado, métodos autorizados, CriterioDeDevolucion	validar métodos de pago	los valores provienen de decisión del negocio (supuestos)	
Producto (referencia)	Categoría, requiere serie, precio de lista	Describir el artículo vendible	Categoría determina el plazo de devolución	Ninguno

4.3 Qué representa cada elemento en el negocio

Entidades y Value Objects

Elemento	Tipo	Qué representa en TecnoStock	Por qué ese tipo
Apartado	Raíz	La reserva de una pieza para un cliente, con plazo y anticipo.	Tiene folio y ciclo de vida; sigue siendo el mismo apartado aunque cambie de estado.
UnidadInventario	Raíz	La pieza física exacta que se guarda, vende, transfiere o devuelve.	Identidad propia (serie); BR-003 exige rastrearla.
Venta	Raíz	El cobro cerrado que respalda lo entregado.	Folio y consistencia de líneas y pagos.
LineaDeVenta	Entidad	Un renglón de la venta: producto, unidad entregada, serie y precio.	Se distingue dentro de la venta, pero no existe fuera de ella.
Devolucion	Raíz	La solicitud de regresar una unidad y su resolución.	Folio y estados consultables (US02).
PoliticaComercial	Raíz	Las reglas vigentes que el Supervisor define.	Configuración con una sola instancia vigente.
Dinero	VO	Monto con moneda.	Se define por su valor; evita sumar cantidades sin sentido.
NumeroDeSerie	VO	Identificador impreso de una pieza.	Dos series iguales son la misma serie; no tiene vida propia.
Folio	VO	Número único de comprobante.	Valor sin comportamiento propio.
PagoRecibido	VO	Método y monto cobrados.	Hecho inmutable; permite validar contra métodos autorizados.
VigenciaDeApartado	VO	Desde cuándo y hasta cuándo rige el apartado.	Su regla (HaVencido) depende solo de sus valores.
MotivoDeDevolucion	VO	Causal y detalle que captura el vendedor (US08).	Describe, no tiene identidad.

Elemento	Tipo	Qué representa en TecnoStock	Por qué ese tipo
DictamenDeDevolucion	VO	Resultado de evaluar el criterio estándar.	Es un resultado calculado e inmutable.
DecisionDeSupervisor	VO	Autorización o denegación documentada (usuario, motivo, fecha).	Solo puede crearse con rol Supervisor (RN-SUP-002).
Operador	VO	Empleado con su rol en el momento de la operación.	Se usa como dato; la gestión de usuarios queda fuera.

Servicio de dominio, políticas y eventos

Elemento	Qué representa en TecnoStock	Evidencia
ConversionDeApartado (Domain Service)	La operación «el cliente liquida su apartado»: verifica vigencia, liquida el apartado, entrega la unidad reservada y crea la venta con el anticipo. No pertenece a un solo aggregate porque modifica tres.	US07, RN-APT-002/003
CriterioDeDevolucion y ReglaDeDevolucion (Policy)	El criterio estándar como lista de reglas intercambiables: ReglaDeComprobanteVigente (RN-DEV-001) y ReglaDePlazoPorCategoria (RN-DEV-002).	US10, US16, H-02
ApartadoCreado	Se formalizó una reserva; interesa a comprobante y a la confirmación al cliente.	US03, US12
ApartadoVencido	Se cumplió el plazo sin liquidar y la unidad se liberó; interesa a quien debe avisar.	US20
VentaCerrada	Se completó una venta; dispara el comprobante.	US12
DevolucionDerivadaASupervisor	Una devolución incumplió el criterio y espera decisión; interesa al Supervisor.	US10, US13
DevolucionAceptada	Una devolución procede; dispara comprobante y reingreso de la unidad.	BR-003, US12
DevolucionRechazada	El Supervisor negó una excepción; queda registro para consulta.	US02, US13

4.4 Diagrama del mapa de dominio

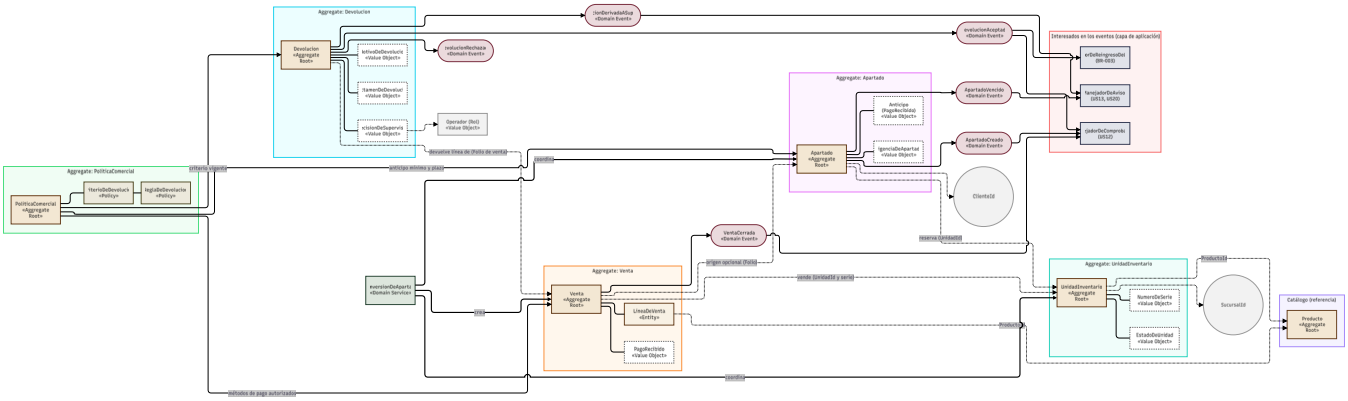


Figura 1

Figura 1. Mapa de dominio: aggregates, Value Objects, servicio de dominio, eventos e interesados.

4.5 Qué protege el dominio y qué no

Las reglas que describen lo que el negocio permite, calcula o protege están en el dominio: bloqueo de unidad apartada, anticipo mínimo, vigencia, serie obligatoria, criterio de devolución y rol del Supervisor. Fuera del dominio quedan la autenticación, la persistencia, el envío de avisos y la impresión del comprobante, que son detalles de infraestructura.

4.6 Reglas de consistencia entre aggregates

- **Misma transacción:** CU-01, CU-02 y CU-03 modifican Unidad y Apartado/Venta juntos, porque una invariante lo exige: no vender lo apartado y no dejar un apartado vigente sobre una unidad vendida.
- **Eventual:** el reingreso de la unidad tras una devolución aceptada, el comprobante y los avisos se ejecutan por eventos tras confirmar la transacción (ADR-003).
- **Unicidad de serie (RN-INV-002)** no puede protegerla un solo aggregate; se verifica en el repositorio dentro de la transacción y se respalda con un índice único en infraestructura.

4.7 Decisiones dudosas o descartadas

Opción considerada	Decisión	Razón
Inventario por producto y sucursal con colección de unidades	Descartada	Aggregate muy grande: dos ventas de unidades distintas del mismo producto competirían por el mismo bloqueo.

Opción considerada	Decisión	Razón
Contador de existencias por producto	Descartada	Pierde la serie y no puede expresar «la unidad apartada» (RN-APT-003, BR-003).
Apartado contiene a la UnidadInventario	Descartada	Mezclaría ciclos de vida distintos y el aggregate cambiaría por razones de inventario.
Cliente como aggregate	Descartada (H-08)	El BRD no define sus datos ni reglas; basta ClienteId.
Comprobante como aggregate	Descartada	Es un documento derivado del hecho; su identidad es el Folio de la operación y se emite por evento.
EvaluadorDeDevolucion como servicio de dominio	Descartada	La evaluación depende solo del criterio; se resolvió como Policy (CriterioDeDevolucion) dentro de PoliticaComercial.
ApartadoConvertidoEnVenta como evento	Descartada	Redundante con VentaCerrada, que indica su apartado de origen.
Eventos UnidadApartada y UnidadLiberada	Descartados	La unidad cambia en la misma transacción; ningún interesado externo los necesita.

5. Service Layer (Etapa 4)

Los Application Services exponen cada caso de uso como un método con intención de negocio. Cada método sigue el mismo ciclo: obtiene el operador, la hora y el folio mediante puertos; inicia la unidad de trabajo; recupera objetos con repositorios; invoca comportamiento del dominio; guarda; confirma; y traduce el resultado. **Ninguna decisión de negocio se toma en el servicio.**

5.1 Servicios y métodos

Servicio	Método (caso de uso)	Entrada	Colaboradores	Salida	Reglas que delega (y dónde viven)
ServicioDeApartados	ApartarProducto (CU-01)	ApartarProductoCommand: producto, sucursal, cliente, anticipo	Repos de Unidades, Productos, Políticas, Apartados; UoW; Reloj; Folios; Contexto	Resultado con ConfirmacionDeApartadoDto (folio, fecha límite, saldo)	Anticipo mínimo y método autorizado: Apartado.Crear / PoliticaComercial. Unidad disponible: UnidadInventario.Apartar. Sin existencia: repositorio (NoEncontrado/Si

Servicio	Método (caso de uso)	Entrada	Colaboradores	Salida	Reglas que delega (y dónde viven)
					nExistencia).
ServicioDeApartados	ConvertirApartadoEnVenta (CU-03)	ConvertirApartadoEnVentaCommand: folio, pago del saldo, serie capturada	Repos de Apartados, Unidades, Productos, Políticas, Ventas; ConversionDeApartado; UoW	Resultado con la venta, o Rechazo ApartadoVencido	Vigencia, saldo, serie y creación de la venta: ConversionDeApartado, Apartado, UnidadInventario y Venta.
ServicioDeVentas	RegistrarVenta (CU-02)	RegistrarVentaCommand: sucursal, líneas (producto, serie), pagos	Repos de Unidades, Productos, Políticas, Ventas; UoW; Reloj; Folios; Contexto	Resultado con VentaCerrada	Unidad apartada y sucursal: UnidadInventario.Vender. Serie obligatoria, método de pago y pago completo: Venta.Cerrar / LineaDeVenta.
ServicioDeDevoluciones	SolicitarDevolucion (CU-04)	SolicitarDevolucionCommand: folio de venta, producto, serie, motivo	Repos de Ventas, Productos, Políticas, Devoluciones; UoW; Reloj; Folios	Resultado con folio y estado (Aceptada o EnRevision)	Comprobante válido: repositorio de Ventas. Plazo y criterio: CriterioDeDevolucion. Estado resultante: Devolucion.Solicitar.
ServicioDeDevoluciones	AutorizarDevolucionExcepcional (CU-05)	AutorizarDevolucionExcepcionalCommand: folio, autoriza, motivo	Repo de Devoluciones; UoW; Reloj; Contexto	Resultado con estado final	Rol Supervisor: DecisionDeSupervisor. Transición válida: Devolucion.Resolver.

5.2 Secuencia de orquestación de un método

Ejemplo, ConvertirApartadoEnVenta: (1) obtener operador, hora y folio; (2) Iniciar(); (3) obtener Apartado, Unidad, Producto y Política; (4) llamar a ConversionDeApartado.Convertir; (5) agregar la Venta si fue exitosa y guardar Apartado y Unidad; (6) Confirmar(); (7) devolver Resultado. El detalle completo con alternativas está en la Figura 4.

5.3 Revisión obligatoria

Pregunta	Resultado
¿El nombre del método expresa una acción del negocio?	Sí: ApartarProducto, RegistrarVenta, ConvertirApartadoEnVenta, SolicitarDevolucion, AutorizarDevolucionExcepcional, tomados del glosario.
¿El método coordina o decide?	Coordina. Las condiciones (vigencia, anticipo, serie, rol, plazo) están en Apartado, UnidadInventario, Venta, Devolucion, DecisionDeSupervisor y CriterioDeDevolucion. El único condicional del servicio es persistir lo que el dominio devolvió.
¿Depende de interfaces o de detalles de base de datos?	Solo de interfaces: repositorios, IUnidadDeTrabajo, IReloj, IGeneradorDeFolios e IContextoDeUsuario.
¿Hay inicio y fin transaccional comprensible?	Sí: Iniciar() al entrar y Confirmar() o Descartar() al salir (sección 7).
¿Los errores se explican en términos del negocio?	Sí: ReglaDeNegocioViolada lleva un MotivoDeRechazo del glosario (UnidadApartada, ApartadoVencido, SoloSupervisorAutoriza…) y TraductorDeRechazos genera el mensaje y la alternativa sugerida.

6. Repository (Etapa 5)

6.1 Repositorios (en términos del dominio)

Repository	Necesidad del dominio	Métodos propuestos	Objeto que recupera o guarda
IRepositorioDeApartados	Recuperar un apartado por su folio para convertirlo, vencerlo o cancelarlo.	Obtener(folio); Agregar(apartado); Guardar(apartado)	Apartado
IRepositorioDeUnidades	Hallar la unidad correcta para apartar, vender, convertir o devolver.	Obtener(unidadId); ObtenerPorSerie(serie); ObtenerUnidadDisponible(producto, sucursal); ObtenerUnidadParaVenta(producto, sucursal, serieCapturada); Guardar(unidad)	UnidadInventario
IRepositorioDeVentas	Verificar el comprobante de compra y registrar ventas.	ObtenerPorFolio(folio); Agregar(venta)	Venta
IRepositorioDeDevoluciones	Seguir la devolución desde la solicitud hasta la resolución.	Obtener(folio); Agregar(devolucion); Guardar(devolucion)	Devolucion
IRepositorioDePoliticas	Conocer los valores	ObtenerPoliticaVigente()	PoliticaComerci

Repository	Necesidad del dominio	Métodos propuestos	Objeto que recupera o guarda
	vigentes.		al
IRepositorioDeProductos	Conocer categoría, si requiere serie y precio.	Obtener(productoId)	Producto

Contrato común. Los métodos que buscan un único objeto no devuelven nulo: señalan ReglaDeNegocioViolada con motivo NoEncontrado (o ComprobanteNoValido, SinExistenciaDisponible). Así, cualquier implementación es sustituible sin cambiar el comportamiento esperado (principio L). No hay método Eliminar: las unidades y operaciones no se borran, por trazabilidad (BR-003).

Implementación en infraestructura (sin base de datos por ahora)

- Primera etapa: implementaciones en memoria (Repositorio…EnMemoria) para ejercitar el diseño y las pruebas del dominio.
- Etapa posterior: implementaciones relacionales que traduzcan agregados a tablas. Requisitos que ya se conocen: índice único sobre el número de serie activo (RN-INV-002) y campo de versión en UnidadInventario para control de concurrencia optimista.
- Las consultas de existencias por sucursal no pasan por estos repositorios: usan los lectores del lado de consulta (sección 8).

7. Unit of Work

Aspecto	Propuesta
Dónde inicia	En la primera línea de cada método del servicio (IUnidadDeTrabajo.Iniciar()).
Qué cambios incluye	CU-01: Apartado nuevo y UnidadInventario → Apartada. CU-02: Venta nueva y unidades → Vendida. CU-03: Apartado → Convertido, unidad → Vendida y Venta nueva (o, si venció, Apartado → Vencido y unidad → Disponible). CU-04: Devolución nueva. CU-05: Devolución resuelta.
Cuándo termina	Confirmar() tras guardar todos los agregados; la UoW publica entonces los eventos que los aggregates registraron.
Qué ocurre ante un error	Una ReglaDeNegocioViolada o un ConflictoDeConcurrencia provoca Descartar(): no se guarda nada ni se publica ningún evento. La excepción es el rechazo con efecto (apartado vencido), que sí se confirma.
Concurrencia	Si dos operadores intentan vender la misma unidad, la versión de UnidadInventario hace fallar al segundo con ConflictoDeConcurrencia, que se

Aspecto	Propuesta
	traduce como «la unidad fue modificada por otra operación» (bloqueo atómico del BRD).
Fuera de la transacción	Comprobante, aviso al Supervisor o al cliente y reingreso de unidades devueltas, en transacciones propias disparadas por eventos.

8. CQRS básico (Etapa 6)

Se separa el cambio de estado (Commands, que pasan por el dominio) de la consulta (Queries, que leen directamente y devuelven DTO). No se crea una base de datos separada: ambos lados pueden compartir el almacén físico, pero los modelos son distintos.

Tipo	Nombre	Datos de entrada	Handler	Resultado	Justificación
Command	ApartarProducto	producto, sucursal, cliente, anticipo	ServicioDeApartados.ApartarProducto	ConfirmacionDeApartadoDto	Cambia estado de Apartado y UnidadInventario (CU-01).
Command	RegistrarVenta	sucursal, líneas, pagos	ServicioDeVentas.RegistrarVenta	Resultado con folio de venta	Cambia estado de Venta y unidades (CU-02).
Command	ConvertirApartadoEnVenta	folio, pago del saldo, serie	ServicioDeApartados.ConvertirApartadoEnVenta	Resultado con folio de venta o Rechazo	Modifica tres aggregates (CU-03).
Command	SolicitarDevolucion	folio de venta, producto, serie, motivo	ServicioDeDevoluciones.SolicitarDevolucion	Folio y estado	Crea Devolucion según criterio (CU-04).

Tipo	Nombre	Datos de entrada	Handler	Resultado	Justificación
Command	AutorizarDevolucionExcepcional	folio, autoriza, motivo	ServicioDeDevoluciones.AutorizarDevolucionExcepcional	Estado final	Resuelve una devolución en revisión (CU-05).
Query	ConsultarDisponibilidadEnSucursal	producto, sucursal	ConsultasDeInventario.ConsultarDisponibilidadEnSucursal	DisponibilidadDeProductoDto	US04: cantidad disponible actualizada; solo lectura.
Query	ConsultarDisponibilidadEnOtrasSucursales	producto, sucursal actual	ConsultasDeInventario.ConsultarDisponibilidadEnOtrasSucursales	Lista de DisponibilidadPorSucursalDto	US05: alternativa cuando no hay existencia local.
Query	ConsultarEstadoDeDevolucion	folio	ConsultasDeDevoluciones.ConsultarEstadoDeDevolucion	EstadoDeDevolucionDto (estado y motivo)	US02: el cliente ve aceptada, rechazada o en revisión.
Query	ConsultarCriterioDeDevolucionVigente	—	ConsultasDeDevoluciones.ConsultarCriterioDeDevolucionVigente	CriterioDeDevolucionDto	US16: consulta uniforme para todas las sucursales.

- **Handler.** Cada método del servicio es el handler de su Command o Query; no se agregan clases handler ni un mediador, para no sumar indirección sin beneficio en cinco sucursales (ADR-004).
- **Dependencias.** Los handlers de comandos dependen de repositorios y de IUnidadDeTrabajo; los de consulta, solo de ILectorDeInventario e ILectorDeDevoluciones.

- **DTO de lectura.** DisponibilidadDeProductoDto y los demás se diseñan para pantalla (producto, sucursal, cantidad) y no reutilizan los aggregates ni el modelo de escritura.
- **Sin separación.** El catálogo de productos y la política comercial se mantienen con una operación simple de lectura/actualización: no justifican Commands propios en este alcance.

9. Revisión con SOLID (Etapa 7)

La guía define cinco preguntas de revisión. Se aplicaron al diseño completo y los hallazgos se registran después con evidencia, impacto y cambio realizado.

Principio	Pregunta de revisión	Evidencia que deben buscar
S	¿Cada clase o servicio tiene una razón principal para cambiar?	Responsabilidades mezcladas, clases extensas, servicios que hacen de todo.
O	¿Pueden agregar una variante de regla sin editar una cadena creciente de condiciones?	Condicionales que deben modificarse por cada nueva política.
L	¿Las implementaciones pueden sustituirse sin cambiar el comportamiento esperado?	Subtipos que rechazan operaciones o alteran contratos.
I	¿Cada consumidor depende solo de las operaciones que utiliza?	Interfaces grandes con métodos irrelevantes para algunos clientes.
D	¿La aplicación y el dominio dependen de abstracciones?	Dependencias directas de SQL, frameworks o servicios externos.

La revisión se aplicó a la agrupación inicial considerada (un único servicio con todas las operaciones, repositorio genérico y reglas con condicionales) y produjo los cambios que reflejan los diagramas finales. Cada hallazgo indica evidencia, impacto y cambio realizado.

ID	Principio	Evidencia	Impacto	Cambio realizado
S-1	S	Un solo «ServicioDeTecnoStock» agrupaba apartados, ventas, devoluciones y consultas.	Cuatro razones de cambio; el servicio crecería con cada historia.	Se separó en ServicioDeApartados, ServicioDeVentas y ServicioDeDevoluciones, y las consultas en ConsultasDeInventario y ConsultasDeDevoluciones.
S-2	S	Venta.Cerrar generaba el folio, el comprobante y descontaba inventario.	Mezcla regla de cobro, formato de documento e inventario.	El folio viene de IGeneradorDeFolios; el comprobante lo emite ManejadorDeComprobantes al recibir VentaCerrada; la unidad cambia en UnidadInventario.Vender.

ID	Principio	Evidencia	Impacto	Cambio realizado
S-3	S	Devolucion evaluaba plazo, comprobante y notificaba al Supervisor.	Cambiar la política obligaría a editar el aggregate.	La evaluación pasó a CriterioDeDevolucion; el aviso lo hace ManejadorDeAvisos por evento.
O-1	O	La validación de devolución era un if por categoría y por causal.	Cada regla nueva (H-02) editaría una cadena de condiciones.	ReglaDeDevolucion como interfaz con ReglaDeComprobanteVigente y ReglaDePlazoPorCategoria; una regla nueva es una clase nueva.
O-2	O	Métodos de pago autorizados escritos como comparaciones en Venta.	Cada cambio de política exigiría recompilar el dominio.	Los métodos autorizados, el anticipo mínimo y los plazos son datos de PoliticaComercial.
L-1	L	Se proponían UnidadSerializada y UnidadNoSerializada como subclases; la segunda rechazaría la operación de serie.	Subtipo que rompe el contrato de la superclase.	Una sola UnidadInventario con NumeroDeSerie opcional; Producto.requiereSerie decide cuándo es obligatorio.
L-2	L	Un repositorio podía devolver nulo y otro lanzar excepción.	Los servicios se comportarían distinto según la implementación.	Contrato único: un objeto inexistente produce ReglaDeNegocioViolada con motivo del glosario.
I-1	I	IRepositorio genérico con Obtener, Agregar, Eliminar y Listar para todos.	Los servicios dependerían de métodos que no usan; Eliminar contradice la trazabilidad.	Seis interfaces por aggregate con solo lo que cada caso necesita; lectura en ILectorDeInventario e ILectorDeDevoluciones.
I-2	I	IUnidadDeTrabajo exponía todos los repositorios.	Cada servicio recibiría acceso a repositorios que no necesita.	IUnidadDeTrabajo solo tiene Iniciar, Confirmar y Descartar; cada servicio recibe los repositorios que usa.
D-1	D	El servicio llamaba a DateTime.Now y creaba folios directamente.	Imposible probar el vencimiento de un apartado sin esperar siete días.	Puertos IReloj, IGeneradorDeFolios e IContextoDeUsuario inyectados; el dominio recibe «ahora» como parámetro.
D-2	D	Los servicios instanciaban repositorios concretos.	La aplicación quedaría atada a la persistencia.	Los servicios dependen de interfaces; el dominio no referencia infraestructura. Las flechas apuntan hacia el dominio.

Clases que realizaban varias cosas y su separación

Clase con varias responsabilidades	Responsabilidades mezcladas	Separación final
ServicioDeTecnoStock	Apartados, ventas, devoluciones y consultas	3 servicios de escritura + 2 de consulta
Venta	Reglas de cobro, folio, comprobante, inventario	Venta (reglas) + IGeneradorDeFolios + ManejadorDeComprobantes +

Clase con varias responsabilidades	Responsabilidades mezcladas	Separación final
		UnidadInventario
Devolucion	Evaluar criterio, decidir estado, notificar	Devolucion + CriterioDeDevolucion + ManejadorDeAvisos
Apartado	Vigencia, bloqueo de unidad, aviso de vencimiento	Apartado + UnidadInventario + ManejadorDeAvisos

10. Diagramas UML y ADR (Etapa 8)

El código Mermaid de todos los diagramas está en el Anexo A y en los archivos .mmd entregados, para importarlos en draw.io (Organizar → Insertar → Avanzado → Mermaid). Los nombres coinciden con el glosario y entre diagramas.

10.1 Diagrama de clases (completo, en dos láminas)

Por legibilidad, la misma arquitectura se divide en dos láminas con los mismos nombres; en draw.io pueden colocarse en un solo lienzo. Diferenciación por capa: **dominio** en tono arena, **aplicación** en verde grisáceo e **infraestructura** en gris azulado.

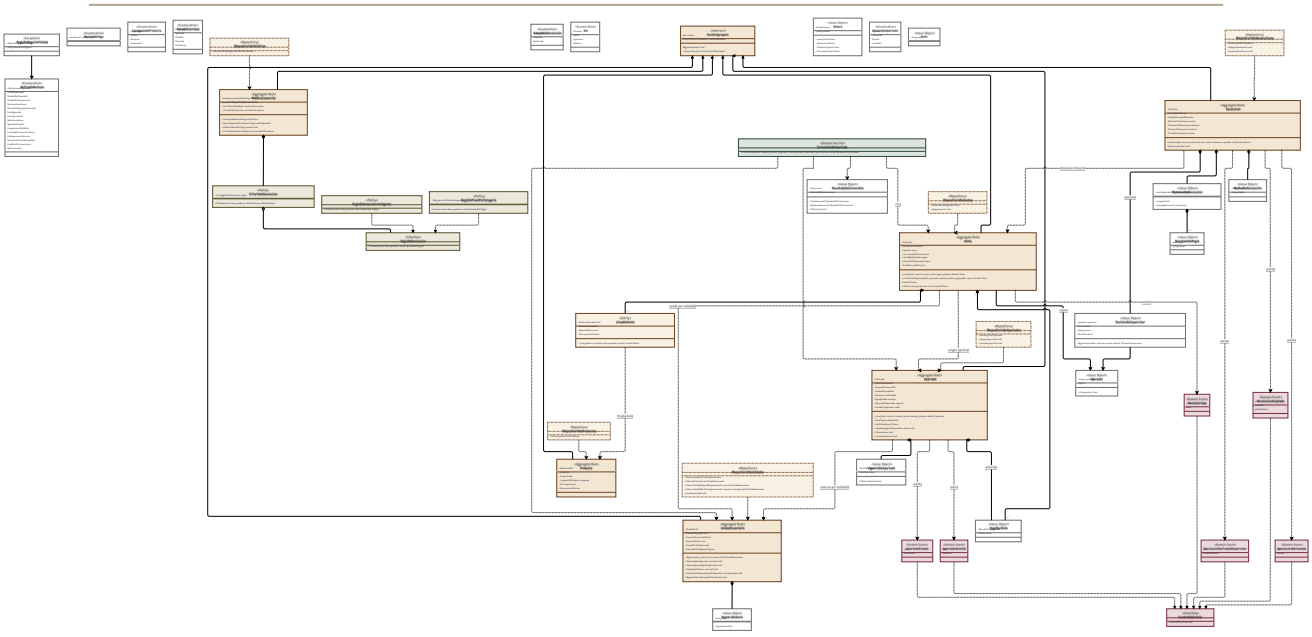


Figura 2

Figura 2. Diagrama de clases del dominio.

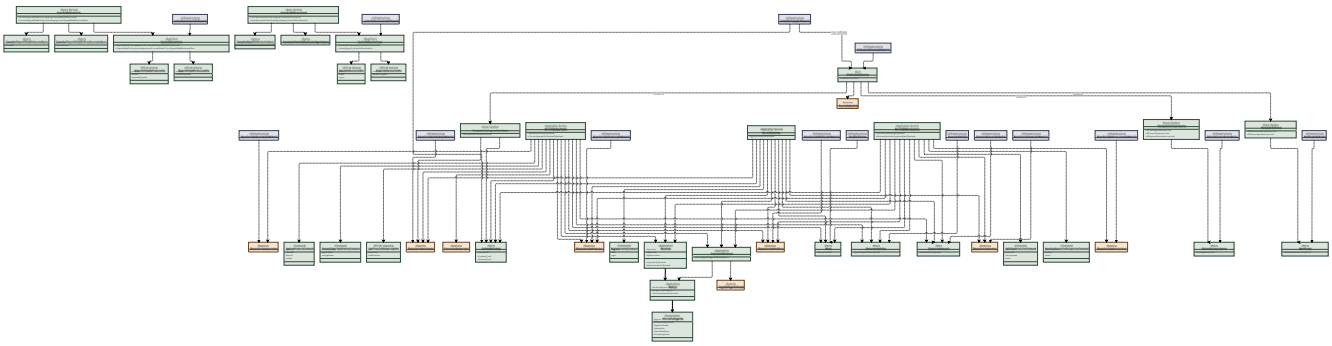


Figura 3

Figura 3. Diagrama de clases de aplicación e infraestructura, con Commands, Queries y DTO.

10.2 Diagramas de secuencia

Figura 4 – flujo principal con alternativas (CU-03): convertir un apartado vigente en venta; incluye el rechazo con efecto (apartado vencido) y el rollback por pago inválido.

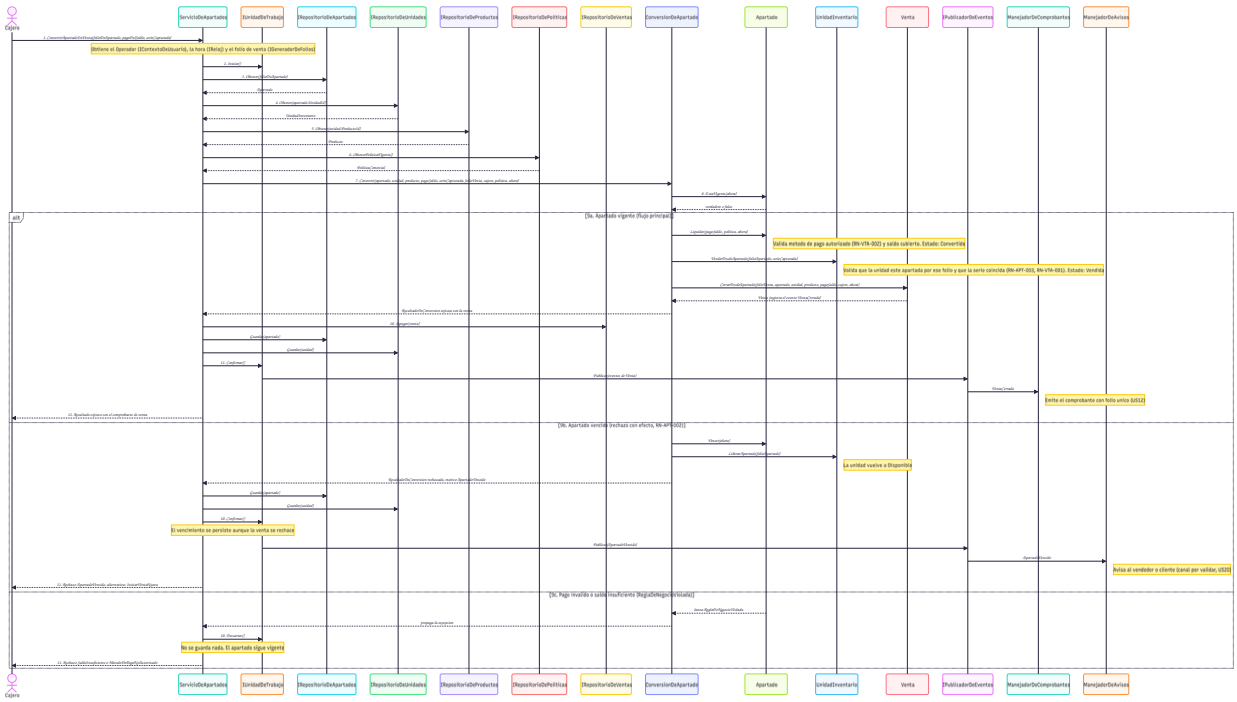


Figura 4

Figura 4. Secuencia de ConvertirApartadoEnVenta.

Figura 5 – flujo con regla y excepción (CU-04 y CU-05): una devolución fuera de criterio se deriva al Supervisor, quien autoriza o deniega.

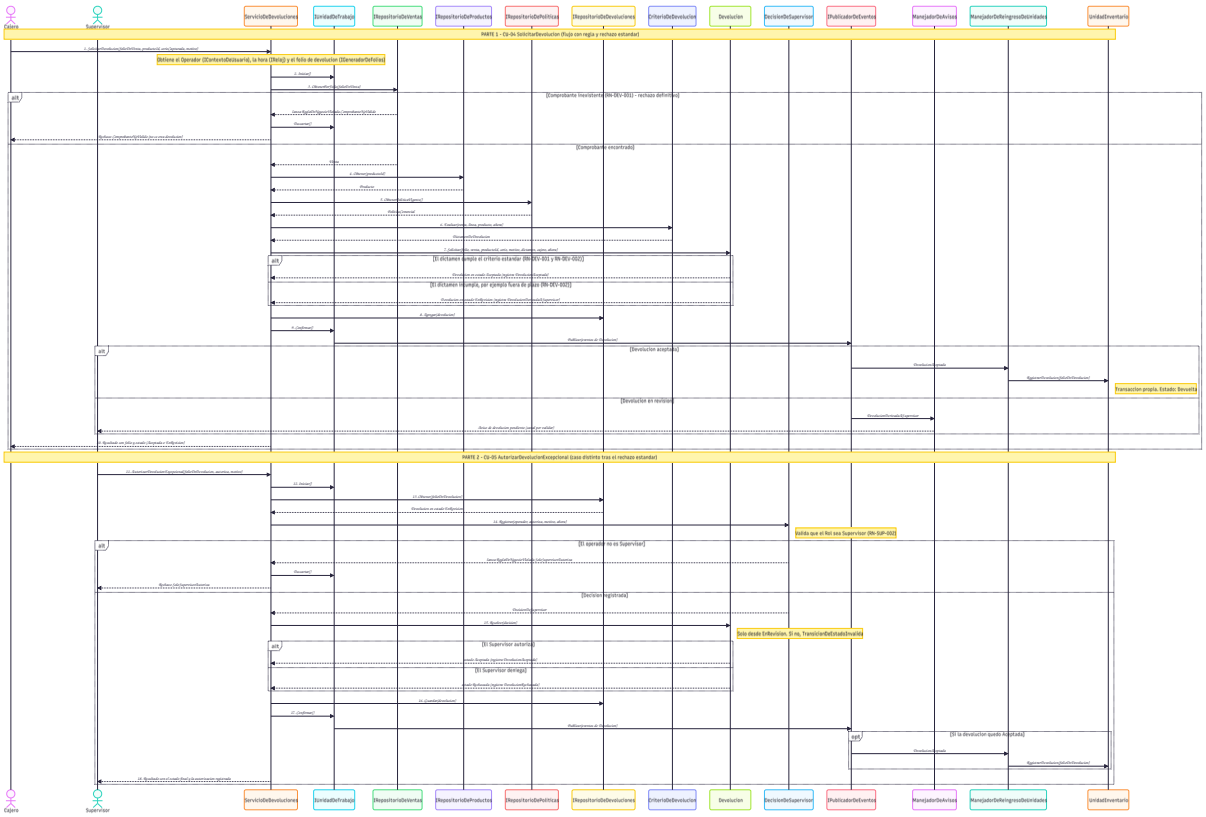


Figura 5

Figura 5. Secuencia de SolicitarDevolucion y AutorizarDevolucionExcepcional.

10.3 Trazabilidad de los componentes

CU	US	BR	Reglas	Componentes principales
CU-01	US01, US03, US11, US12	BR-004, BR-002	RN-APT-001*, 002*, 003; RN-VTA-002	ServicioDeApartados.ApartarProducto, Apartado, UnidadInventario, PoliticaComercial, ApartadoCreado
CU-02	US04, US06, US09, US12	BR-001, BR-002, BR-003	RN-VTA-001, 002; RN-APT-003; RN-INV-002	ServicioDeVentas.RegistrarVenta, Venta, LineaDeVenta, UnidadInventario, VentaCerrada
CU-03	US07, US12, US20	BR-004, BR-003	RN-APT-002*, 003; RN-VTA-001, 002	ServicioDeApartados.ConvertirApartadoEnVenta, ConversionDeApartado, Apartado, Venta, ApartadoVencido
CU-04	US10, US08, US12, US02	BR-005, BR-003	RN-DEV-001, 002*	ServicioDeDevoluciones.SolicitarDevolucion, Devolucion, CriterioDeDevolucion, DevolucionAceptada,

CU	US	BR	Reglas	Componentes principales
				DevolucionDerivadaASupervisor
CU-05	US13, US16	BR-006, BR-005	RN-SUP-001, 002	ServicioDeDevoluciones.AutorizarDevolucionExcepcional, DecisionDeSupervisor, DevolucionRechazada

11. Architecture Decision Records

ADR-001. Domain Model como patrón de lógica de negocio

Campo	Contenido
Título	Domain Model como patrón de lógica de negocio
Estado	Aceptada
Contexto	Reglas cruzadas entre apartado, unidad, venta y devolución; estados con transiciones; parámetros aún supuestos; problema de venta accidental de apartados y de series sin trazabilidad.
Alternativas	Transaction Script (reglas duplicadas, estados dispersos); Table Module (reglas entre tablas sin dueño); Domain Model.
Decisión	Domain Model para los comandos (CU-01 a CU-05) y Transaction Script solo para lecturas. Table Module descartado.
Consecuencias	Beneficio: invariantes en un solo lugar y pruebas sin infraestructura. Costo: más clases, mapeo de persistencia y disciplina para no mover reglas a los servicios. Trabajo posterior: validar los supuestos con el stakeholder.
Trazabilidad	BR-003, BR-004, BR-005; RN-APT-002/003, RN-INV-003, RN-DEV, RN-SUP; US06, US07, US13; H-01 a H-08.

ADR-002. Límites de aggregates: UnidadInventario, Apartado, Venta y Devolucion separados

Campo	Contenido
Título	Límites de aggregates: UnidadInventario, Apartado, Venta y Devolucion separados
Estado	Aceptada
Contexto	Hay que bloquear una unidad concreta sin bloquear todo el producto, y conservar la identidad de la serie en venta, tránsito y devolución.
Alternativas	Inventario por producto-sucursal con unidades dentro; contador por producto; Apartado que contiene a la unidad; aggregates separados que se refieren por identidad.
Decisión	Aggregates separados con referencia por identidad (UnidadId, Folio). UnidadInventario es raíz propia con control de versión.

Campo	Contenido
Consecuencias	Beneficio: menor contención y trazabilidad por unidad. Costo: la unicidad de serie (RN-INV-002) se verifica en el repositorio y con índice único; CU-01 y CU-03 modifican dos o tres aggregates en una transacción.
Trazabilidad	BR-003; RN-APT-003, RN-INV-002; US06, US09, US17.

ADR-003. Service Layer con una Unidad de Trabajo por caso de uso y eventos tras confirmar

Campo	Contenido
Título	Service Layer con una Unidad de Trabajo por caso de uso y eventos tras confirmar
Estado	Aceptada
Contexto	Hay que evitar ventas sobre stock apartado (consistencia fuerte) y a la vez desacoplar comprobante, avisos y reingreso de unidades devueltas.
Alternativas	Todo en una sola transacción; todo por eventos; transacción por caso de uso con efectos secundarios por eventos.
Decisión	Cada método de servicio abre una UoW, modifica en conjunto solo los aggregates ligados por una invariante y confirma. Los efectos secundarios se ejecutan por eventos tras Confirmar().
Consecuencias	Beneficio: consistencia donde importa y desacoplamiento donde no. Costo: consistencia eventual en reingreso de unidades devueltas (requiere reintento o bandeja de salida) y manejo de fallos de los manejadores.
Trazabilidad	BR-003, BR-004; RN-APT-003; US12, US13, US20.

ADR-004. CQRS básico sin base de datos separada; el handler es el método del servicio

Campo	Contenido
Título	CQRS básico sin base de datos separada; el handler es el método del servicio
Estado	Aceptada
Contexto	Las consultas de disponibilidad, estado de devolución y criterio vigente no necesitan reconstruir aggregates y requieren un modelo de lectura distinto.
Alternativas	Un solo modelo para lectura y escritura; CQRS completo con bus y base separada; CQRS básico con handlers como métodos.
Decisión	Commands por los Application Services; Queries por ConsultasDeInventario y ConsultasDeDevoluciones, que usan lectores y devuelven DTO. Sin bus ni mediador.
Consecuencias	Beneficio: lecturas simples y rápidas, escritura protegida. Costo: dos caminos que

Campo	Contenido
	mantener alineados; si el volumen crece se puede evolucionar a un almacén de lectura sin cambiar los contratos.
Trazabilidad	BR-001; US02, US04, US05, US16.

ADR-005. Políticas parametrizables y reglas de devolución como estrategias

Campo	Contenido
Título	Políticas parametrizables y reglas de devolución como estrategias
Estado	Aceptada
Contexto	El BRD deja valores como supuestos (anticipo 20%–30%, 7 días, 15–30 días por categoría) y BR-005 menciona condiciones sin regla definida.
Alternativas	Constantes en el código; condicionales por categoría; PoliticaComercial con ReglaDeDevolucion intercambiables.
Decisión	Los valores viven en PoliticaComercial y el criterio de devolución es una lista de ReglaDeDevolucion.
Consecuencias	Beneficio: el Supervisor puede cambiar valores y se pueden añadir reglas sin editar código existente. Costo: una abstracción más y la necesidad de definir cómo se versiona la política.
Trazabilidad	RN-APT-001/002, RN-DEV-001/002, RN-VTA-002, RN-SUP-002; US16; H-02.

ADR-006. Modelo de rechazo: excepción de dominio, resultado con derivación y rechazo con efecto

Campo	Contenido
Título	Modelo de rechazo: excepción de dominio, resultado con derivación y rechazo con efecto
Estado	Aceptada
Contexto	Los rechazos tienen significados distintos: unos no dejan rastro, otros dejan la devolución en revisión y otros deben guardar el vencimiento de un apartado.
Alternativas	Excepciones para todo; códigos de retorno para todo; combinar según el tipo de rechazo.
Decisión	Violación de regla: ReglaDeNegocioViolada (rollback). Derivación: estado válido EnRevision. Rechazo con efecto: ResultadoDeConversion rechazada y Confirmar(). TraductorDeRechazos genera el Rechazo con alternativa sugerida.
Consecuencias	Beneficio: errores expresados en términos del negocio y cada caso alterno es explícito.

Campo	Contenido
	Costo: el equipo debe distinguir los tres tipos al crear reglas nuevas.
Trazabilidad	US06, US10, US13, US20; RN-APT-002, RN-DEV-001/002, RN-SUP-002.

12. Validación de la propuesta (Etapa 9)

Se recorrió cada caso de uso sobre los diagramas: qué componente recibe la solicitud, dónde se valida cada regla, qué objetos cambian, cómo se persisten los cambios y qué resultado se devuelve (secuencias de las Figuras 4 y 5).

Validación	Pregunta	Resultado del equipo
Trazabilidad	¿Cada componente importante responde a una necesidad, regla o caso de uso?	Sí. La matriz del Anexo B relaciona BR, RN, US, CU, componentes y ADR; los componentes sin caso de uso (p. ej. estados EnTransito) se marcan como extensión.
Lenguaje	¿Los mismos términos significan lo mismo en BRD, UML, código y ADR?	Sí. Los nombres provienen del glosario (4.1) y se repiten en diagramas, código base y ADR; los términos nuevos están marcados.
Responsabilidad	¿Las reglas permanecen en el dominio o en el patrón elegido con una justificación?	Sí. Anticipo, vigencia, serie, bloqueo de unidad, criterio de devolución y rol del Supervisor están en el dominio; los servicios solo coordinan (sección 5.3).
Consistencia	¿Los diagramas de clase y secuencia representan la misma arquitectura?	Sí. Los participantes y métodos de las Figuras 4 y 5 existen en las Figuras 2 y 3 con el mismo nombre.
Persistencia	¿El dominio evita depender directamente de SQL, ORM o detalles de infraestructura?	Sí. El dominio solo declara interfaces de repositorio; las implementaciones están en infraestructura (en memoria por ahora).
Transacción	¿El límite de la operación y su manejo de error están definidos?	Sí. Una unidad de trabajo por caso de uso, con Confirmar y Descartar (sección 7); el rechazo con efecto está definido en ADR-006.
IA	¿Cada aporte de IA fue revisado y su uso está registrado?	Sí. Documento aparte «Bitácora de IA» con prompts, errores detectados y decisiones del equipo.

13. Entregables y repositorio

La siguiente tabla relaciona los entregables de la guía con lo que contiene esta entrega. El código base y la bitácora se entregan en archivos propios.

Entregable	Contenido mínimo (guía)	Estado y ubicación
01 Informe de arquitectura	Portada, alcance, comparación de patrones, mapa de dominio, Service Layer, Repository, Unit of Work, CQRS, revisión SOLID, ADR y conclusiones.	Este documento (exportar a PDF para la entrega).
02 Diagrama de dominio y clases	Archivo fuente y exportación legible en PDF o PNG.	Fuentes .mmd en diagrams/fuente. Exportación desde draw.io pendiente del equipo.
03 Diagramas de secuencia	Dos archivos fuente y sus exportaciones.	Fuentes .mmd en diagrams/fuente. Exportación pendiente del equipo.
04 Matriz de trazabilidad	Relación entre necesidades, reglas, historias, casos de uso, componentes y decisiones.	Anexo B de este informe.
05 Bitácora de IA	Prompts, respuestas relevantes, errores detectados, correcciones y decisiones humanas.	Documento aparte: Bitacora_IA_Unidad3.docx.
06 Código base	Estructura de carpetas y código mínimo que represente la arquitectura propuesta.	Carpeta src/ y tests/ del repositorio (sección 13.2); paquete tecnostock-arquitectura.zip.
07 Repositorio GitHub	Todos los archivos organizados, historial de commits y README con instrucciones de revisión.	Enlace en la sección 13.1; README incluido en el paquete.
08 Presentación y defensa	Exposición breve donde todos los integrantes puedan explicar y defender decisiones.	A cargo del equipo; las preguntas de defensa se responden en la sección 14.

13.1 Enlace al repositorio de GitHub

Campo	Contenido
Repositorio	https://github.com/monroydiego/TecnoStock
Rama de entrega	main
Integrantes con acceso	Eduardo Alejandro Acevedo Liuskos, Emma Patricia Brown Sánchez, Diego Monroy García
Cómo revisarlo	Abrir README.md; leer docs/01_informe_arquitectura; ejecutar «./run_tests.sh» desde la raíz (instrucciones en el README).

13.2 Estructura de carpetas y relación con los patrones elegidos

La estructura sigue la sugerida por la guía (docs, diagrams, src, tests y README). Dentro de src se refleja la arquitectura elegida: una carpeta por capa (dominio,

aplicación, infraestructura) y, dentro del dominio, una carpeta por aggregate. La dependencia va siempre hacia el dominio.

```
tecnostock-arquitectura/
├── README.md
├── run_tests.sh                    Ejecuta las pruebas (unittest)
├── docs/
│   ├── 01_informe_arquitectura/    Informe (docx/pdf)
│   ├── adr/                        ADR-001 a ADR-006
│   ├── trazabilidad/               Matriz de trazabilidad
│   └── ia/                         Bitácora de IA y prompts
├── diagrams/
│   ├── fuente/                     Archivos .mmd (Mermaid)
│   └── exportaciones/              PNG o PDF exportados de draw.io
├── src/tecnostock/
│   ├── dominio/
│   │   ├── compartido/             Value Objects, errores, eventos, raíz de aggregate
│   │   ├── catalogo/               Producto
│   │   ├── politicas/              PoliticaComercial
│   │   ├── inventario/             UnidadInventario + IRepositorioDeUnidades
│   │   ├── apartados/              Apartado, ConversionDeApartado + repositorio
│   │   ├── ventas/                 Venta, LineaDeVenta + repositorio
│   │   └── devoluciones/           Devolucion, CriterioDeDevolucion, reglas, decisión
│   ├── aplicacion/
│   │   ├── servicios/              Application Services (escritura, Commands)
│   │   ├── consultas/              Query Services y DTO de lectura
│   │   ├── puertos/                Unidad de trabajo, reloj, folios, contexto, publicador
│   │   ├── eventos/                Manejadores de eventos de dominio
│   │   ├── comandos.py             Commands
│   │   └── resultado.py            Resultado, Rechazo y traductor de rechazos
│   ├── infraestructura/
│   │   ├── persistencia_en_memoria/    Repositorios, lectores y unidad de trabajo
│   │   └── servicios/              Reloj, folios, contexto de usuario, publicador
│   └── composicion.py              Ensamble de dependencias
└── tests/
    ├── dominio/                    Invariantes de aggregates
    └── aplicacion/                 Los cinco casos de uso y sus rechazos
```

Carpeta	Patrón o decisión que representa	Evidencia
dominio/ (una carpeta por aggregate)	Domain Model: cada aggregate protege sus invariantes	ADR-001, ADR-002
dominio/*/repositorio	Repository: interfaces en términos del negocio, sin SQL	Sección 6
dominio/politicas y dominio/devoluciones/reglas	Políticas intercambiables (principio O)	ADR-005
aplicacion/servicios	Service Layer: un método por caso de uso, solo coordina	Sección 5
aplicacion/puertos	Inversión de dependencias (principio D) y Unit of Work	Secciones 7 y 9
aplicacion/consultas	CQRS básico: lado de lectura con DTO	ADR-004
aplicacion/eventos	Domain Events: efectos tras confirmar la transacción	ADR-003
infraestructura/	Detalles reemplazables; en memoria porque no se desarrolla base de datos	Sección 6
tests/	Escenarios de aceptación de los cinco casos de uso (Gherkin de la U2 como referencia)	Sección 12

13.3 Código base

El código base es mínimo y ejecutable: representa los aggregates con sus invariantes, los Application Services de los cinco casos de uso, los repositorios y la unidad de trabajo en memoria, el lado de consulta y los manejadores de eventos. Está escrito en Python 3 sin dependencias externas (pruebas con unittest). Las pruebas recorren los flujos normales y de rechazo de CU-01 a CU-05. No incluye base de datos, interfaz ni autenticación.

13.4 Historial de commits sugerido

La guía pide historial de commits; se sugiere una secuencia que muestre la evolución del diseño:

- docs: agregar BRD, historias de usuario y guía de la actividad (insumos U2).
- docs: comparación de patrones y ADR-001.
- feat(dominio): value objects, errores y aggregates con invariantes.
- feat(aplicacion): servicios de los cinco casos de uso y unidad de trabajo.
- feat(infraestructura): repositorios y lectores en memoria.
- test: escenarios de aceptación de CU-01 a CU-05.

- docs: diagramas, revisión SOLID, bitácora de IA y matriz de trazabilidad.

14. Conclusiones y defensa

14.1 Conclusión General del Equipo

- La arquitectura propuesta es coherente con el negocio: el Domain Model protege los estados y las invariantes donde el BRD identifica los problemas (venta de apartados, series, criterios de devolución).
- La Service Layer solo coordina, los repositorios hablan en términos del glosario y la Unidad de Trabajo delimita cada operación con su manejo de error.
- El CQRS básico separa cinco comandos de cuatro consultas sin infraestructura adicional.
- La revisión SOLID produjo once hallazgos con cambios concretos; las clases con varias responsabilidades se separaron.
- Quedan ocho preguntas para el stakeholder (H-01 a H-08), en especial el destino del anticipo, los abonos parciales y las condiciones de estado y causal en devoluciones. El diseño las deja como puntos de extensión y no inventa reglas.

14.2 Conclusiones Personales

Diego Monroy García 2135198

A lo largo del desarrollo de este proyecto, mi principal aprendizaje ha sido comprender que la arquitectura de software no se trata únicamente de dibujar diagramas o aplicar patrones de moda, sino de tomar decisiones conscientes para resolver problemas reales del negocio y proteger la integridad de los datos. Al analizar la problemática de TecnoStock en particular los errores críticos como vender productos apartados o perder el rastro de los números de serie comprendí el valor práctico de implementar un Domain Model. Pero en este caso entendí que encapsular las reglas en entidades y agregados específicos (como UnidadInventario o Apartado) es la única forma de garantizar que una regla comercial no se pase por alto cuando el sistema crezca.

Eduardo Alejandro Acevedo Liuskos 2142139

La verdad es que este proyecto con TecnoStock fue un reto bastante interesante. Al principio, viendo todos los problemas que tenían en las tiendas como vender por accidente productos que ya estaban apartados, no saber exactamente qué número de serie tenía cada cliente, o las discusiones por el criterio de las devoluciones, todo parecía un problema. Lo que más me gustó del trabajo fue darme cuenta de cómo una buena arquitectura de software de verdad le soluciona la vida al negocio y a las personas que trabajan ahí. Pasar de esas reglas de negocio sueltas a un diseño donde el sistema solito detiene las equivocaciones (como no dejarte vender una laptop que alguien ya apartó) le quita un peso enorme de encima tanto a los vendedores como al supervisor. Obviamente no todo quedó resuelto mágicamente; nos quedaron algunas dudas en el camino como qué hacer con el dinero del anticipo cuando vence un apartado o a dónde va a parar físicamente un producto devuelto, pero lo valioso es que la estructura que armamos es lo suficientemente flexible para que, en cuanto el negocio tome una decisión, podamos ajustarlo sin tener que rehacer todo desde cero.

Emma Patricia Brown Sánchez 2135479

El desarrollo de esta actividad permitió estructurar una propuesta arquitectónica coherente para TecnoStock, enfocada en resolver directamente los problemas críticos identificados en el análisis previo, tales como la venta accidental de productos apartados, la falta de trazabilidad en unidades las inconsistencias en las devoluciones, a través de la implementación del modelo de dominio, la separación de responsabilidades en capas y el uso de una unidad de trabajo por caso de uso, el equipo logró asegurar que las reglas del negocio se protejan adecuadamente sin sobrecargar los servicios de aplicación,asimismo, la integración de políticas parametrizables y la revisión bajo los principios SOLID permitieron corregir múltiples deficiencias estructurales, dotando al sistema de flexibilidad ante futuros cambios. Si bien quedaron ciertos aspectos sujetos a validación con los stakeholders como los detalles específicos sobre los anticipos y los destinos físicos de las devoluciones, el diseño final entrega una base sólida, auditable y libre de sobrediseño para el manejo de las operaciones del retail.

14.3 Preguntas para la defensa

Pregunta	Respuesta preparada
¿Qué evidencia del negocio influyó más	Los estados con ciclo de vida del apartado y la unidad (RN-

Pregunta	Respuesta preparada
en la selección del patrón?	APT-002/003) y la operación US07, que involucra tres conceptos a la vez.
¿Qué responsabilidad fue la más difícil de ubicar?	La conversión de apartado en venta: se resolvió con un servicio de dominio (ConversionDeApartado) para no poner reglas en el Application Service ni acoplar los aggregates.
¿Qué regla protege el diseño y qué pasaría en una capa incorrecta?	El bloqueo de la unidad apartada. Si estuviera solo en la interfaz o en el servicio, otro camino de venta podría omitirlo (la venta accidental del BRD).
¿Qué cambió después de la revisión con SOLID?	Se dividió el servicio único, se reemplazaron condicionales por reglas intercambiables, se evitó una jerarquía de unidades que violaba L y se redujo la interfaz de la unidad de trabajo.
¿Qué propuesta de la IA rechazaron o corrigieron?	Inventario por producto y sucursal, contador de existencias, Cliente y Comprobante como aggregates, y la decisión de sugerir alternativas dentro del servicio. Detalle en la bitácora.
¿Qué consecuencia negativa aceptaron?	Más clases y mapeo de persistencia, y consistencia eventual en el reingreso de unidades devueltas.

15. Bitácora de IA

15.1 Herramienta y contexto

Aspecto	Detalle
Herramienta	Claude (Anthropic), en una sesión con el proyecto «Componentes de lógica de Negocio» adjunto (BRD, User Stories, guía de la actividad, imagen AF03 y formato del documento).
Insumos entregados a la IA	BRD y User Stories de TecnoStock (U2), guía de la Unidad 3, imagen AF03 con los cuatro entregables, instrucciones de formato (.docx).
Datos excluidos	No se envió información ajena al caso académico.
Salidas de la IA	Informe de arquitectura (.docx), cinco diagramas Mermaid, repositorio base con pruebas y esta bitácora.
Responsabilidad	El equipo revisó el informe y lo aprobó con observaciones de formato (iteración 2). Cada integrante debe poder defender las decisiones sin leer una respuesta generada.

15.2 Ciclo de trabajo aplicado

Paso	Acción	Evidencia
1	Preparar contexto	Se adjuntaron BRD, US, guía y AF03; la IA leyó los documentos

Paso	Acción	Evidencia
		completos antes de diseñar, y registró 8 hallazgos de continuidad (informe, 1.2).
2	Solicitar propuesta	Mensaje 1 (iteración 1). La IA debía justificar el patrón con evidencia y declarar supuestos: aparecen marcados con asterisco y en los hallazgos H-01 a H-08.
3	Comparar	Se contrastó cada propuesta con los IDs del BRD (BR, RN) y de las US; ver tabla de validación (sección 5).
4	Corregir	Se descartaron alternativas de dominio y se reubicaron reglas (sección 6). El equipo corrigió formato y estructura en la iteración 2.
5	Validar	Trazabilidad (Anexo B del informe), consistencia entre diagramas y pruebas automáticas de los cinco casos de uso (22 pruebas, todas pasan).
6	Registrar	Esta bitácora, más los archivos fuente (.mmd), el informe y el repositorio.

15.3 Bitácora de iteraciones

Las iteraciones 1 y 2 corresponden a mensajes realmente enviados por el equipo (se transcriben completos en la sección 7). Las iteraciones 3 a 8 son pasos internos que la IA ejecutó dentro de la iteración 1 siguiendo los formatos de prompt de la guía; no fueron mensajes separados y se identifican así para no atribuir al equipo prompts que no envió.

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Error o supuesto	Decisión del equipo
1	Producir la arquitectura completa de la actividad (casos de uso, patrón, dominio, Service Layer, Repository, UoW, CQRS, SOLID, UML, ADR).	Mensaje 1 (sección 7.1).	Informe .docx de 37 páginas y cinco diagramas Mermaid.	La IA contó 20 US y no 15 (H-01). Supuestos H-02 a H-08. Las tablas del informe no seguían exactamente las columnas de la guía.	Se aceptó la información. Se pidió corregir formato y estructura (iteración 2).
2	Alinear el informe con el formato de la guía y completar entregables	Mensaje 2 (sección 7.2).	Informe reestructurado, matriz de trazabilidad, sección de código base y GitHub, repositorio	La versión 1 omitió código base, matriz y bitácora por interpretar «solo AF03 y la guía» de forma restrictiva.	Se agregaron los entregables 04, 05, 06 y 07.

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Error o supuesto	Decisión del equipo
	faltantes.		base y esta bitácora.		
3	Comparar patrones (paso interno de 1).	Formato de la guía «Patrones» aplicado a los 15 RN y los CU seleccionados.	Tabla comparativa con evidencia; Domain Model con lectura en Transaction Script.	Inicialmente se consideró EvaluadorDeDevolucion como servicio de dominio.	Se resolvió como política (CriterioDeDevolucion); ADR-001 y ADR-005.
4	Proponer el modelo de dominio (paso interno).	Formato «Dominio» aplicado al glosario y reglas del BRD.	Aggregates, Value Objects, eventos y servicio de dominio ConversionDeApartado.	Se propusieron inventario por producto-sucursal, contador de existencias, Cliente y Comprobante como aggregates.	Rechazadas (informe 4.7, ADR-002). Estado «Devuelta» queda como supuesto H-06.
5	Diseñar Service Layer y repositorios (paso interno).	Formato «Service Layer» aplicado a CU-01 a CU-05.	Servicios por área, handler como método, repositorios con términos del negocio.	Una versión preliminar decidía en el servicio qué hacer sin existencia y cuándo vencía un apartado.	Reglas movidas al dominio (contrato del repositorio, ConversionDeApartado); el servicio solo coordina.
6	Revisar con SOLID (paso interno).	Formato «SOLID» aplicado al diseño.	11 hallazgos (S, O, L, I, D) con evidencia, impacto y cambio.	Servicio único, repositorio genérico, jerarquía de unidades y uso directo de la hora del sistema.	Corregidos todos; ver informe sección 9.
7	Generar diagramas (paso interno).	Formato «Diagramas» con la arquitectura validada.	Cinco archivos Mermaid compilados sin error.	Un diagrama de clases único resultaba ilegible.	Se dividió en dos láminas con los mismos nombres; secuencias con alternativas.
8	Código base (paso interno de 2).	Estructura de la guía (docs, diagrams, src, tests, README).	Repositorio con dominio, aplicación, infraestructura en memoria y 22 pruebas.	pytest no estaba disponible en el entorno.	Se usó unittest de la biblioteca estándar.

15.4 Verificaciones técnicas realizadas

Verificación	Cómo se hizo	Resultado
Sintaxis de diagramas	Compilación de los cinco archivos .mmd con el compilador de Mermaid.	Sin errores; imágenes generadas y revisadas.
Validez del .docx	Validador de esquema Office y conversión a PDF para revisar las páginas.	Validación aprobada.

Verificación	Cómo se hizo	Resultado
Consistencia entre diagramas	Comparación de participantes y métodos de las secuencias con las láminas de clases.	Todos los nombres existen en las láminas 2 y 3.
Comportamiento esperado	22 pruebas automáticas: los cinco casos de uso, rechazos, rollback, rechazo con efecto, eventos y consultas.	22 de 22 aprobadas.

15.5 Validación humana de los aportes de la IA

Aporte de la IA	Contraste con la Unidad 2	Resultado
Selección de CU-01 a CU-05	Cada caso aparece en US01–US13 y en las reglas RN-APT, RN-VTA, RN-DEV, RN-SUP.	Aceptado
Domain Model como patrón	Estados (RN-APT-002/003), interacción (US07) y parámetros por rango (supuestos del BRD).	Aceptado
Estado «Devuelta» de la unidad	El BRD no define el destino tras una devolución (H-06).	Aceptado como supuesto pendiente
Inventario por producto-sucursal	Contradice «unidad apartada» (RN-APT-003) y trazabilidad (BR-003).	Rechazado
Comprobante y Cliente como aggregates	El BRD no define sus reglas ni datos (H-08).	Rechazado
Reglas dentro del Application Service	Contradice la guía (el servicio solo coordina).	Corregido
Valores 20%, 7 días, 15 y 30 días	Marcados como supuesto en el BRD.	Aceptados como parámetros configurables

15.6 Errores detectados y correcciones

- **Cantidad de User Stories:** se mencionaron 15; el documento tiene 20. Se trabajó con 20 y se registró (H-01).
- **Formato de tablas:** el informe inicial no usaba las columnas de la guía. Se reestructuró (iteración 2).
- **Entregables omitidos:** matriz de trazabilidad, código base, repositorio y bitácora. Se agregaron.
- **Reglas en la capa equivocada:** la decisión de vencimiento y la elección de alternativas estaban en el servicio; se movieron al dominio y a un traductor de rechazos.

- **Herramienta de pruebas:** pytest no estaba disponible; se usó unittest.

Propuestas de la IA rechazadas o corregidas (pregunta de defensa)

Inventario por producto-sucursal, contador de existencias por producto, Cliente y Comprobante como aggregates, EvaluadorDeDevolucion como servicio de dominio, jerarquía de unidades serializadas y no serializadas, repositorio genérico y unidad de trabajo que exponía todos los repositorios.

15.7 Prompts utilizados

15.7.1 Mensaje 1 (enviado por el equipo)

Vamos a realizar avances de un proyecto, en el contexto vienen las indicaciones tanto como formato de los documentos .docx e instrucciones particulares para el desarrollo de la actividad.

Instrucciones especiales | formato: tipografía en Times New Roman y que no se utilicen colores genéricos en el documento.

Instrucciones particulares: Utilizar los mismos US y BRD. Unidad 3: cómo organizar las necesidades del desarrollo. 01 - Qué User Stories elegir, seleccionar cuál modelo quede mejor para el negocio establecido. Seleccionar entre 5 casos de uso (tenemos 15), donde incluyamos operaciones normales, donde la operación se pueda rechazar y tomar un caso distinto, justificar con evidencias por qué se seleccionó el patrón de diseño. 02 - Explicar qué representa cada elemento dentro del negocio de lo que seleccionamos de los patrones. 03 - Service Layer: ubicar cada método de esa capa; cada caso de uso únicamente coordina el trabajo, no contiene regla de negocio. Los repositorios deben hablar con términos de negocio. Utilizar CQRS básico, diferenciar responsabilidades, NO desarrollar base de datos por el momento. 5 principios del SOLID: revisar si está acorde, detectar clases que realicen varias cosas y separar responsabilidades. 04 - Creación de diagramas UML.

Objetivo: realizar una arquitectura coherente; se va a evaluar por qué se tomó la decisión de que esa arquitectura es la mejor y el conocimiento del negocio.

Entregables: únicamente los de la imagen 'AF03.png' y los solicitados en la guía; cuando se trate de diagramas compartir el código en formato Mermaid para exportarlo manualmente con draw.io.

15.7.2 Mensaje 2 (enviado por el equipo)

Hice la revisión del documento (informe) y está bien, solo hay unos aspectos que quiero corregir.

Información: la información es correcta, el único fix que veo es el formato y estructura tal cual lo pide la guía, ya que el documento final contiene la información correcta pero no con el formato y estructura de datos de las tablas de la guía.

Acerca de: mencionas que no agregaste la parte del código base y la matriz de trazabilidad. Agreguemos un apartado para el entregable de código base donde pueda incluir el link de GitHub, y hacer un documento aparte sobre la bitácora de la IA, con los prompts que se utilizaron; cubrir todos sus puntos.

GitHub: la estructura de carpetas tiene que ser acorde al contexto del proyecto y sus patrones escogidos; la estructuración también viene en el punto donde habla de la IA.

15.7.3 Formatos de prompt de la guía aplicados al caso (reproducibles)

Son los formatos de la guía completados con los datos de TecnoStock. Describen los pasos internos de las iteraciones 3 a 7 y permiten que el equipo los reproduzca y compare resultados.

Tema	Prompt
Patrones	Con base en las reglas RN-VTA, RN-APT, RN-INV, RN-DEV y RN-SUP del BRD, los supuestos con rangos (anticipo, vigencia, plazos) y los casos de uso CU-01 a CU-05 de TecnoStock, compara Transaction Script, Table Module y Domain Model. Declara tus supuestos y no elijas una opción hasta mostrar sus consecuencias.
Dominio	Revisa este glosario y estas reglas del BRD de TecnoStock [pegar glosario y RN]. Propón candidatos a entidades, Value Objects, Aggregates y Domain Events. Para cada candidato, indica la evidencia utilizada y las dudas que requieren validación humana.
Service Layer	A partir de los casos de uso CU-01 a CU-05 [pegar tabla de CU], propone firmas de Application Services. Separa coordinación de reglas de dominio y explica cualquier responsabilidad dudosa.
SOLID	Analiza este diseño de TecnoStock [pegar servicios, repositorios y aggregates] y señala posibles violaciones SOLID. No reescribas todo; identifica evidencia, riesgo y alternativas para que el equipo decida.
Diagramas	Convierte esta arquitectura validada [pegar descripción] en un borrador Mermaid. No agregues clases, reglas o relaciones que no aparezcan en la descripción.

15.8 Declaración de uso responsable y pendientes

- Cada aporte de la IA fue revisado contra el BRD y las User Stories antes de integrarse.
- Los supuestos y datos faltantes están marcados y se convierten en preguntas para el stakeholder (H-01 a H-08).
- Pendiente del equipo: agregar a esta bitácora sus propias iteraciones posteriores (por ejemplo, cambios hechos al repositorio o a los diagramas en draw.io) con prompt, respuesta, error y decisión.

Pendiente del equipo: cada integrante debe ensayar la defensa de las decisiones sin apoyo de la IA.